Step 1 — The Report Shape, the Executor, and the Compiler — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the shapes and the executor
- Hint 2 — the citation logic

Hint 1 — the shapes and the executor

Three models and one loop:

- `SubQuestions(sub_questions: List[str])` — the planner's output.
- `Finding(sub_question, answer, source)` — one researched sub-question.
- `Report(question, findings)` — the compiled result.

`execute_plan(sub_questions, search)` runs `search(sub_question)` (which returns `(text, source)`) for each one and builds a `Finding`. The trick that makes it testable: `search` is **passed in** — a fake in tests, the RAG lookup in the pipeline.

Hint 2 — the citation logic

`format_report` numbers sources in first-seen order and de-duplicates them, so two findings from the same document share one number:

```
sources = []                                # first-seen order, no duplicates
for finding in report.findings:
    if finding.source not in sources:
        sources.append(finding.source)

for finding in report.findings:
    number = sources.index(finding.source) + 1   # its citation number
    print(f"- {finding.sub_question}")
    print(f"  {finding.answer} [{number}]")
# then print "Sources:" and each source with its [number]
```

The key property: if two findings cite `rag.md`, they both show `[1]`, and the Sources list has `rag.md` once. Test it with a fake `search` where two sub-questions map to the same source, and check both cite `[1]`. The whole file set is in the Solution.